

Tracking Performance Drift in Large Language Models Across Successive Version Updates

Abstract

Large language models (LLMs) are increasingly deployed as continuously updated services rather than static artifacts. This deployment model creates a distinctive evaluation problem: a model identifier may remain constant while the underlying parameters, post-training procedures, safety policies, routing mechanisms, system prompts, or inference configurations change. Consequently, a system can exhibit measurable performance and behavioral drift across successive releases even when users perceive it as the “same” model. This paper synthesizes the emerging literature on longitudinal evaluation of LLMs and develops a methodological framework for tracking such drift. It distinguishes version drift from conventional data drift and concept drift, identifies the interaction between benchmark saturation, contamination, prompt sensitivity, stochasticity, and evaluator instability, and reviews current approaches including holistic evaluation, dynamic benchmarks, contamination-resistant test sets, human-preference evaluation, and automated LLM-based judging. Particular attention is given to empirical evidence that GPT-3.5 and GPT-4 changed substantially between March and June 2023, to temporal evaluation mechanisms such as NEO-BENCH and LiveCodeBench, and to recent work formalizing behavioral trajectories across model generations. The paper argues that longitudinal monitoring should move beyond scalar benchmark deltas toward multidimensional drift profiles that jointly track accuracy, calibration, instruction fidelity, safety, robustness, output structure, latency, and cost. A rigorous protocol should preserve immutable evaluation cohorts, maintain rolling temporally novel cohorts, repeat stochastic tests, report uncertainty and effect sizes, archive exact prompts and outputs, and separate genuine capability change from evaluation artifacts. The paper concludes by identifying research gaps in causal attribution of drift, cross-provider version equivalence, continuous benchmark governance, user-specific risk monitoring, and statistically principled release gates. Longitudinal evaluation is presented not as an adjunct to conventional benchmarking but as a necessary component of trustworthy lifecycle management for rapidly evolving LLM systems.

Keywords: large language models; performance drift; model versioning; longitudinal evaluation; benchmark stability; temporal drift; reproducibility; model monitoring; LLM evaluation; benchmark contamination

1. Introduction

The evaluation paradigm for machine learning has traditionally assumed that a model is a relatively stable object. A research paper reports results for a specified checkpoint; subsequent experiments compare against that checkpoint; and reproduction is expected to be possible provided that the model, data, code, and evaluation procedure are held constant. Modern LLM services disrupt this assumption. Providers increasingly expose models through APIs in which the nominal service can evolve through parameter updates, fine-tuning, safety interventions, revised system prompts, routing policies, inference optimization, and other changes. In such environments, the object of evaluation is not simply a model but a continually changing socio-technical system.

This creates an important distinction between **static model evaluation** and **longitudinal performance monitoring**. Static evaluation asks whether a particular model performs well on a specified benchmark at a point in time. Longitudinal evaluation asks whether the performance and behavioral properties of the same deployed service remain stable, improve, or deteriorate across successive updates. The latter question is operationally important because changes can occur in opposite directions across capabilities. An update may improve factual question answering while worsening instruction adherence, increase refusal rates on benign prompts while strengthening safeguards, or improve coding correctness while altering output formatting that downstream software depends upon. The well-known longitudinal study by Chen, Zaharia, and Zou (2024) demonstrated precisely this pattern: GPT-3.5 and GPT-4 changed substantially between their March and June 2023 versions, with some capabilities improving and others deteriorating.

The problem is broader than simple regression testing. A measured change between release t and release $t + 1$ may reflect at least five different phenomena: (1) genuine change in model capability; (2) a change in instruction-following or alignment behavior; (3) changes in the evaluation distribution; (4) benchmark contamination or saturation; or (5) stochastic and evaluator-induced measurement noise. Distinguishing these causes is essential. A benchmark score change without such attribution can lead to both false alarms and false reassurance.

The literature on LLM evaluation has expanded rapidly. HELM established a multi-metric framework emphasizing standardized conditions, broad scenario coverage, and dimensions beyond accuracy (Liang et al., 2022; Bommasani et al., 2023). MMLU created a widely used broad knowledge and reasoning benchmark, while subsequent work such as MMLU-Pro has attempted to reduce noise and increase discriminative power (Hendrycks et al., 2021; Wang et al., 2024). More recent benchmarks such as LiveBench and LiveCodeBench respond directly to contamination and temporal staleness by introducing new problems over time.

However, existing evaluation infrastructure still tends to compare models horizontally rather than track them longitudinally. Leaderboards often emphasize the latest score, implicitly treating a release as a point estimate of capability rather than one observation in a temporal process. This paper therefore focuses on the research question:

How should performance drift in LLMs be defined, measured, attributed, and governed across successive version updates?

The contribution is a scholarly synthesis and methodological framework rather than a new empirical benchmark. The paper first reviews the conceptual foundations of LLM evaluation and temporal drift, then examines current approaches, proposes a unified longitudinal monitoring architecture, identifies limitations and research gaps, and outlines future research directions.

2. Background and Related Work

2.1 From scaling to lifecycle evaluation

Research on language-model progress has historically centered on scaling. Kaplan et al. (2020) demonstrated predictable relationships between model size, data, compute, and language-model loss, while Hoffmann et al. (2022) showed that compute-optimal training changes the balance between

parameter count and training data. These studies explain why successive generations can improve systematically, but they do not imply that every update is uniformly superior on every downstream dimension.

Downstream evaluation itself is multidimensional. MMLU evaluates broad knowledge and reasoning over 57 subjects, whereas GSM8K targets mathematical word-problem solving and SWE-bench evaluates software-engineering tasks grounded in real GitHub issues (Hendrycks et al., 2021; Cobbe et al., 2021; Jimenez et al., 2024). These benchmarks operationalize different slices of capability; improvement on one cannot be presumed to imply improvement on another.

This observation motivates the notion of a **capability vector** rather than a scalar model score. If release v_t has capability vector

$$\mathbf{P}_t = (P_{\text{accuracy}}, P_{\text{reasoning}}, P_{\text{safety}}, P_{\text{faithfulness}}, P_{\text{instruction}}, P_{\text{robustness}}, P_{\text{efficiency}}, \dots),$$

then version quality is more realistically characterized by trajectories in a multidimensional space:

$$\Delta \mathbf{P}_t = \mathbf{P}_{t+1} - \mathbf{P}_t.$$

A release can therefore be an improvement according to one aggregate metric while representing a regression in a safety-critical or application-specific dimension.

2.2 Holistic evaluation

HELM provided one of the clearest foundations for multidimensional evaluation. Liang et al. (2022) proposed a taxonomy of scenarios and desiderata and evaluated models using metrics including accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency. The framework was designed to standardize evaluation across models and to expose trade-offs obscured by single-score leaderboards. The peer-reviewed concise report subsequently emphasized that model behavior should be evaluated across multiple scenarios and metrics rather than by a narrow capability measure (Bommasani, Liang, & Lee, 2023).

The same logic applies temporally. If the evaluation vector is multidimensional, longitudinal monitoring should examine the trajectory of every important dimension rather than merely comparing overall benchmark averages. This is particularly important because updates often modify alignment and instruction-following behavior as well as core task competence.

2.3 Benchmark saturation and contamination

Benchmark scores are meaningful only if the benchmark remains sufficiently discriminative and independent of training data. The growing concern over contamination has produced extensive methodological research. Deng et al. (2024) proposed methods for investigating benchmark contamination in both open and proprietary LLMs, while Chen et al. (2025) surveyed the transition from static to dynamic benchmarking and emphasized the absence of standardized evaluation criteria for dynamic benchmarks.

LiveCodeBench operationalizes the dynamic principle particularly clearly. It continuously collects newly released programming problems and partitions them by release date, allowing researchers to test models

against items that post-date likely training cutoffs. Its authors report that time-segmented evaluation can reveal contamination and that model rankings can vary substantially across code-related capabilities.

The implication for drift tracking is critical: **a stable score on a stale benchmark is not necessarily stable capability**. A model may appear unchanged simply because the evaluation set is saturated, memorized, or insensitive to the behavioral dimension being monitored.

2.4 Temporal drift in language and deployment environments

Not all temporal drift originates from model updates. Language itself evolves. Zheng, Ritter, and Xu (2024) showed that temporal drift between training data and later language can degrade model performance, particularly for emerging neologisms. Their NEO-BENCH experiments found that models with later knowledge cutoffs generally achieved lower perplexity and better downstream performance on newer linguistic phenomena.

This establishes an important conceptual distinction:

- **Model-version drift:** change caused by modification of the deployed model or its surrounding inference system.
- **Data drift:** change in the distribution of inputs encountered after deployment.
- **Concept drift:** change in the relationship between inputs and desired outputs.
- **Evaluation drift:** change in the benchmark itself or its suitability for measuring the target capability.

A longitudinal evaluation system must distinguish these sources because the same observed performance decline can have very different causes and remedies.

2.5 Evaluation reproducibility and statistical inference

Reproducibility is a prerequisite for meaningful drift measurement. Belz (2021) argued for a quantified framework for reproducibility in NLP and machine learning, highlighting the need to distinguish different forms of reproducibility rather than treating them as a binary property. Statistical testing is equally important. Berg-Kirkpatrick, Burkett, and Klein (2012) demonstrated that metric improvements in NLP cannot be interpreted reliably using simple thresholds, while Dror et al. (2018) provided practical guidance on statistical significance testing for NLP experiments.

For LLM drift, this means that a release-to-release difference should be treated as an estimate with uncertainty rather than as an exact quantity. Repeated sampling, confidence intervals, paired tests, and effect-size reporting are necessary to determine whether a measured regression is larger than expected evaluation variance.

3. Main Technical Discussion: What Constitutes Performance Drift?

3.1 A formal definition

Let M_t denote the deployed system at version t , including model weights and relevant serving configuration. Let D_k be an evaluation dataset for capability k , E_k its evaluation function, and θ the

inference configuration (system prompt, temperature, tool settings, context limits, and other relevant parameters). Then:

$$S_{t,k} = E_k(M_t, D_k, \theta).$$

A basic performance drift measure is

$$\Delta_{t,k} = S_{t+1,k} - S_{t,k}.$$

For comparability across metrics, normalized drift can be expressed as

$$\delta_{t,k} = \frac{S_{t+1,k} - S_{t,k}}{\max(\epsilon, |S_{t,k}|)}.$$

However, this formulation is insufficient when the benchmark itself evolves or when output behavior cannot be reduced to a single scalar. A more comprehensive definition treats drift as a distributional change in outputs:

$$\mathcal{D}_t = d(P(Y | X, M_t), P(Y | X, M_{t+1})),$$

where $d(\cdot, \cdot)$ is an appropriate distributional distance and X is a fixed prompt cohort.

This distinction is useful because two versions can achieve the same task score while generating systematically different answers, formats, refusal patterns, verbosity levels, or reasoning behavior. Chen et al. (2024) demonstrated such behavior drift directly, showing that changes in instruction following could partially explain changes in measured task performance.

3.2 Drift dimensions

A robust framework should monitor at least six categories.

Capability drift. This includes changes in correctness on stable tasks such as mathematics, coding, factual question answering, reasoning, and domain-specific tasks.

Behavioral drift. The model may change its verbosity, refusal behavior, adherence to requested formats, tone, or response structure without an obvious change in nominal task accuracy.

Safety drift. Changes in refusal boundaries, harmful-content handling, jailbreak robustness, or unsafe completion rates may matter even when general capability improves.

Reliability drift. Calibration, consistency across paraphrases, susceptibility to perturbations, hallucination frequency, and confidence-error relationships can change independently of mean accuracy.

Efficiency drift. Latency, token consumption, throughput, and cost can change due to serving or routing modifications even when model capability remains stable.

Interaction drift. In tool-using, retrieval-augmented, or multi-turn systems, the model's performance may depend on changing context windows, retrieval behavior, tools, or orchestrators. The observable system is therefore larger than the underlying neural network.

A practical drift vector can be represented as:

$$\Delta_t = (\Delta A, \Delta B, \Delta S, \Delta R, \Delta E, \Delta I).$$

This representation makes it possible to detect **trade-off drift**, such as simultaneous gains in accuracy and losses in instruction fidelity.

3.3 Release-to-release versus cumulative drift

Adjacent releases are not the only meaningful comparison. Suppose a model family has versions $M_1, \dots, M_T$. A local drift estimate is $\Delta_t = M_{t+1} - M_t$, whereas cumulative drift relative to a baseline is

$$C_t = M_t - M_1.$$

The two metrics answer different questions. Local drift detects regressions introduced by a particular update; cumulative drift shows whether a family has moved materially from its historical behavioral profile.

For operational monitoring, both are necessary. A small but repeated regression in a safety-sensitive behavior could become substantial over many updates even if no individual release triggers an alarm.

3.4 Attribution: model change or measurement change?

A major methodological difficulty is attribution. If performance falls from S_t to S_{t+1} , one cannot automatically conclude that the model became less capable.

Four controls are especially useful.

Immutable benchmark cohorts. A fixed, versioned evaluation set allows direct paired comparisons across releases.

Rolling fresh cohorts. A second evaluation stream should contain newly collected data unavailable to earlier models. Dynamic benchmarks such as LiveBench and LiveCodeBench operationalize this principle.

Prompt perturbation cohorts. Because LLM scores can be sensitive to prompt wording, multiple equivalent formulations should be sampled. MMLU-Pro, for example, was explicitly designed to reduce some instability under prompt variation relative to the original MMLU.

Output-level auditing. Raw prompts and completions should be archived so that a score change can be inspected at the example level. HELM's release of prompts and completions illustrates the value of this practice.

3.5 Measuring uncertainty

Suppose a benchmark yields x_t correct answers out of n paired items for release t . A naïve comparison of percentages can exaggerate apparent drift when n is small or item difficulty is uneven.

A stronger approach is to use paired observations and bootstrap the difference:

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (m_{i,t+1} - m_{i,t}),$$

where $m_{i,t}$ is the item-level score. Repeated bootstrap resampling can produce a confidence interval for $\hat{\Delta}$. Chen et al. (2024) used 95% bootstrap confidence intervals in their longitudinal study, illustrating an appropriate approach for release comparisons.

For categorical outcomes, McNemar-type paired tests can be useful; for continuous or ordinal scores, paired permutation or Wilcoxon procedures may be appropriate, depending on assumptions. Multiple dimensions and many benchmarks create a multiple-testing problem, requiring explicit correction or a pre-registered hierarchy of primary metrics.

The practical goal should not be “detect any difference,” because with sufficiently large evaluation sets almost every release will differ by some amount. Instead, monitoring should combine:

1. statistical significance,
2. effect size,
3. practical significance,
4. persistence across cohorts, and
5. relevance to the application.

4. Current Approaches and Developments

4.1 Fixed-benchmark regression testing

The simplest production strategy is to maintain a “golden set” of prompts representing important user workflows and run every new release against it. This approach is valuable because it measures exactly the application behavior that matters to an organization.

Its weakness is coverage. A fixed set inevitably becomes stale and may become known to training systems. It should therefore be treated as a **regression suite**, not as a complete estimate of general model capability.

4.2 Holistic standardized evaluation

HELM demonstrates how standardized evaluation across common scenarios can reduce comparability problems. Its multi-metric design is especially useful for drift monitoring because it makes regressions visible even when top-line accuracy is unchanged.

A longitudinal version of HELM can be conceptualized as a time-indexed panel:

$$\mathbf{S} = \begin{bmatrix} S_{1,1} & S_{1,2} & \cdots & S_{1,K} & S_{2,1} & S_{2,2} & \cdots & S_{2,K} & \vdots & \vdots & \ddots & \vdots & S_{T,1} & S_{T,2} & \cdots & S_{T,K} \end{bmatrix}.$$

This enables trend estimation, changepoint detection, capability correlation analysis, and identification of recurring trade-offs.

4.3 Dynamic and contamination-resistant benchmarks

Dynamic benchmarks address a fundamental problem: evaluation items can become part of future training corpora. LiveCodeBench continuously adds problems from coding competitions and tags them by release date, enabling time-windowed evaluation against relatively fresh tasks.

More generally, dynamic benchmarks should be maintained as **time-indexed measurement instruments** rather than static datasets. Each item should contain metadata such as publication date, source, suspected contamination window, difficulty, domain, evaluation type, and retirement status.

The 2025 EMNLP survey by Chen et al. argues that the field is moving from static toward dynamic evaluation while still lacking standardized criteria for judging the quality of dynamic benchmarks. This is directly relevant to drift tracking: a benchmark that changes too aggressively can itself generate apparent performance trends.

4.4 Human preference and LLM-as-a-judge evaluation

Open-ended generations are difficult to score with exact-match metrics. Zheng et al. (2023) demonstrated that LLM-based judges can approximate human preferences at useful levels of agreement and introduced MT-Bench and Chatbot Arena as complementary approaches. At the same time, they documented position, verbosity, and self-enhancement biases, showing why automated judges should not be treated as infallible measurement devices.

Recent work continues to emphasize judge reliability. LongJudgeBench, for example, reports that LLM judges remain unstable for long-form evaluation and that rubrics and references do not completely eliminate reliability problems.

For longitudinal evaluation, this creates a second-order drift problem: if the **judge** changes, then an apparent model drift may actually be evaluator drift. A robust system should therefore preserve judge versions, calibration examples, scoring rubrics, and adjudication procedures, and periodically compare automated scores with human evaluations.

4.5 Real-world task distributions

WildBench addresses the gap between synthetic benchmarks and user behavior by selecting challenging tasks from large collections of real human-chatbot interactions. Its design suggests an important role for user-derived evaluation in longitudinal monitoring because changes in model behavior may become visible on authentic requests before they appear on standardized academic datasets.

However, real-user evaluation presents privacy, representativeness, and selection-bias challenges. It should therefore complement rather than replace curated scientific tests.

4.6 Domain-specific and application-grounded testing

The best longitudinal test suite for a medical summarization system is unlikely to be MMLU, and the best test for a code-editing agent is not an ordinary chat benchmark. SWE-bench illustrates the value of evaluating systems on realistic workflows that require multi-file reasoning, repository interaction, and executable validation.

A mature monitoring architecture should thus contain three layers:

- a **general capability layer** for broad comparability,
- a **risk layer** for safety, reliability, and robustness,
- an **application layer** derived from actual user workflows.

5. A Proposed Longitudinal Drift-Tracking Framework

The literature suggests a practical architecture that combines the strengths of fixed and dynamic evaluation.

5.1 Evaluation cohorts

Every deployment should be evaluated on four cohorts.

Cohort A: Immutable regression set. A version-controlled collection of high-value application prompts and standard benchmarks.

Cohort B: Fresh temporal set. Newly authored or newly released tasks selected after the previous model's likely data cutoff.

Cohort C: Behavioral probe set. Prompt families specifically designed to measure instruction fidelity, format compliance, refusals, verbosity, consistency, and sensitivity to paraphrasing.

Cohort D: Production-like sample. Privacy-preserving samples drawn from actual workflows or carefully constructed simulations.

This four-cohort design separates regression detection from generalization measurement and from production monitoring.

5.2 Experimental controls

Each release evaluation should record:

- exact model identifier and provider metadata;
- model snapshot or release date when available;

- system/developer prompts;
- temperature, top- p , seed, and token limits;
- tool configuration and retrieval indices;
- judge model and judge prompt;
- benchmark revision;
- hardware or endpoint metadata when relevant;
- raw outputs;
- timestamps and latency.

Without these controls, a “version comparison” may actually compare different inference pipelines.

5.3 Drift dashboard

A useful dashboard should report at least:

Dimension	Primary measure	Drift indicator
Correctness	Accuracy / exact match / execution success	Paired effect size
Reasoning	Task-specific score	Relative change + CI
Instruction following	Constraint satisfaction	Failure-rate delta
Safety	Unsafe completion/refusal metrics	Risk-weighted delta
Robustness	Performance under perturbation	Stability gap
Hallucination	Unsupported-claim rate	Relative risk
Calibration	ECE/Brier or task-specific calibration	Absolute change
Output format	Schema/format validity	Error-rate delta
Efficiency	Latency/cost/token use	Relative change
Behavioral distribution	Embedding or distribution distance	Drift distance

The dashboard should preserve both the latest comparison and the historical trajectory, because isolated release deltas are less informative than persistent trends.

5.4 Drift severity

A useful governance model is to classify drift into four levels.

Level 0 — statistical noise: change is small and confidence intervals overlap a predefined practical-equivalence region.

Level 1 — detectable drift: statistically reliable change with limited operational impact.

Level 2 — material drift: practically meaningful change affecting user workflows.

Level 3 — critical regression: statistically and operationally significant deterioration in a safety-, compliance-, or business-critical dimension.

This approach prevents organizations from treating every statistically significant change as a release blocker while still making consequential regressions visible.

5.5 Example release decision rule

For a critical metric k , define a tolerance T_k . A release is flagged if

$$\Pr(\Delta_k < -T_k) > 1 - \alpha.$$

For multiple metrics, organizations can use a risk-weighted aggregate:

$$R_t = \sum_{k=1}^K w_k g_k(\Delta_{t,k}),$$

where w_k reflects application importance and g_k transforms a metric change into a standardized loss. The aggregate should not replace raw metric reporting; rather, it acts as a governance layer.

6. Research Challenges and Limitations

6.1 Hidden implementation changes

The most fundamental limitation in proprietary LLM services is observability. Researchers often cannot determine whether a provider changed weights, routing, safety filters, system prompts, retrieval infrastructure, or another layer. Therefore, observed drift may be attributable only to the **deployed service**, not the neural model itself.

This suggests that future research should distinguish **model drift** from **service drift**. Users generally care about the latter, whereas scientific model comparison often focuses on the former.

6.2 Benchmark contamination

Contamination can create artificial stability or apparent progress. Choi et al. (2025) showed the importance of measuring benchmark leakage rather than assuming that public test sets remain independent.

A longitudinal framework must therefore continuously reassess whether its fixed cohort remains uncontaminated. This creates an unavoidable tension: scientific reproducibility favors public benchmarks, while contamination resistance favors hidden or newly generated evaluation items.

6.3 Benchmark saturation

A benchmark can cease to differentiate strong models even when their real-world capabilities continue to diverge. MMLU-Pro was motivated partly by this problem, while LiveCodeBench provides evidence that time-segmented benchmarks can remain more informative for coding models.

The research challenge is to detect saturation before it invalidates trend interpretation. Candidate indicators include reduced variance across models, ceiling effects, declining rank correlation with human evaluation, and diminishing ability to discriminate adjacent generations.

6.4 Prompt and evaluator sensitivity

LLMs can be highly sensitive to prompt formulations, and judge models can themselves introduce biases. Consequently, drift estimates can depend on the evaluation procedure. MMLU-Pro reports reduced sensitivity to prompt variations relative to MMLU, while Zheng et al. (2023) documented several LLM-as-a-judge biases.

This indicates that prompt ensembles and evaluator ensembles should be treated as methodological controls rather than optional extras.

6.5 Non-stationarity of user preferences

For systems optimized through user feedback, “better” is not a stationary target. User preferences, product requirements, and acceptable safety trade-offs can change. A release could be preferable to users while scoring worse on an older academic benchmark—or the reverse.

Therefore, longitudinal evaluation should include both **objective task performance** and **application-defined utility**, while documenting how utility definitions evolve.

6.6 Attribution of causal mechanisms

Observational release comparisons establish that behavior changed but usually cannot establish why. The GPT-4/GPT-3.5 study, for example, identified decreased instruction following as one plausible contributor to observed behavior changes, but did not establish a complete causal chain for every drift.

Controlled ablation is rarely possible with proprietary services. Open-weight model families provide an opportunity to study how specific fine-tuning, reward-model, safety-training, or data changes affect drift.

6.7 Multi-turn and agentic systems

Traditional benchmark evaluation is often single-turn. Production systems increasingly involve multiple turns, tools, retrieval, memory, and external environments. In such systems, small per-turn changes may compound over time. Recent research on conversational robustness has begun to model inconsistency as a temporal event rather than a static score, indicating a direction for longitudinal LLM evaluation.

This suggests that future drift metrics should include trajectory-level quantities such as time-to-failure, cumulative deviation, recovery ability, and failure hazard.

7. Research Gaps and Future Directions

7.1 A standard definition of LLM version identity

There is no universally accepted representation of an LLM release. A future standard should define a machine-readable **version manifest** encompassing model identifier, weights or snapshot hash when available, post-training revision, system prompt, safety policy version, tool stack, retrieval configuration, and inference defaults.

Such a manifest would make longitudinal comparisons far more precise.

7.2 Drift-aware benchmark governance

Current benchmark governance largely asks whether a benchmark is valid. A drift-aware benchmark should additionally ask whether its **measurement properties remain stable over time**.

Research should therefore develop benchmark health metrics covering:

- contamination probability;
- ceiling/saturation;
- item aging;
- domain shift;
- prompt sensitivity;
- inter-rater reliability;
- evaluator consistency;
- correlation with real-world utility.

The 2025 survey of static and dynamic benchmarking identifies the absence of standardized criteria as an important unresolved problem.

7.3 Causal drift decomposition

A central future direction is to decompose observed change into:

$$\Delta_{\text{observed}} = \Delta_{\text{model}} + \Delta_{\text{policy}} + \Delta_{\text{routing}} + \Delta_{\text{prompt}} + \Delta_{\text{data}} + \Delta_{\text{measurement}} + \epsilon.$$

This decomposition need not be perfectly identifiable in proprietary systems; even partial attribution would be useful. Open-weight longitudinal studies could experimentally isolate each component.

7.4 Continuous rather than release-bound monitoring

Version numbers are an administrative construct. User-facing drift may occur without explicit public release announcements, particularly when providers change routing or infrastructure. Hence, monitoring should be **time-triggered as well as release-triggered**.

A production system could evaluate a small sentinel suite daily and escalate to a full suite when distributional change exceeds a threshold. This resembles statistical process control more than conventional benchmark evaluation.

7.5 Distribution-aware drift measurement

Most current metrics are task-level aggregates. Future work should combine:

- task scores,
- output embeddings,
- lexical and structural statistics,
- semantic similarity,
- calibration,
- refusal distributions,
- response-length distributions,
- uncertainty estimates,
- error clusters.

Recent work on behavioral evolution has moved toward response-space representations that compare models using shared prompt banks rather than relying solely on task labels. Qiao, Ding, and Fan (2026) propose multiple response-distribution distances and use them to characterize behavioral trajectories across model families.

This approach is promising because it can reveal changes even when conventional task scores remain flat.

7.6 Human-grounded longitudinal evaluation

LLM-based judges are scalable but can drift or fail in systematic ways. Future monitoring systems should maintain a small human-evaluated calibration panel that periodically validates automated metrics.

The core research problem is not eliminating human evaluation, which is unrealistic at scale, but determining the **minimum human sample needed to detect meaningful judge drift**.

7.7 User-specific and risk-specific drift

A universal drift score is unlikely to be optimal. A medical application, software-development system, educational tutor, and creative-writing assistant have different failure costs.

Future systems should estimate

$$\Delta_{u,k}$$

for user/application u and capability k , producing personalized release-risk profiles. Such a framework would connect academic evaluation to operational model governance.

7.8 Drift as a first-class reproducibility variable

Reproducibility studies traditionally focus on whether a published result can be reproduced. For LLM services, researchers should also record whether the underlying service is still behaviorally equivalent.

A paper published in year t may be perfectly reproducible against an archived model snapshot while becoming impossible to reproduce through the provider's current endpoint. Longitudinal version archives should therefore become part of scientific preservation.

8. Conclusion

Successive updates to large language models should not be assumed to produce monotonic improvement. Empirical evidence already demonstrates substantial behavior and performance changes across relatively short intervals, including cases in which one capability improves while another deteriorates. The central challenge is therefore not simply to determine whether a newer model scores higher, but to determine **how the behavior of a deployed LLM has changed, where the change occurred, whether it is statistically and operationally meaningful, and what caused it.**

The literature points toward several convergent principles. First, longitudinal evaluation must be multidimensional. HELM's emphasis on scenarios and metrics provides an important foundation. Second, fixed benchmarks should be complemented by temporally fresh evaluation to mitigate contamination and benchmark aging; dynamic benchmarks such as LiveCodeBench demonstrate the value of this principle. Third, raw outputs and evaluator configurations must be preserved so that apparent drift can be audited. Fourth, statistical uncertainty must be reported, because not every observed difference is a genuine regression. Fifth, LLM-as-a-judge systems can scale evaluation but must themselves be monitored for bias and reliability. Finally, evaluation should increasingly include realistic user tasks and agentic workflows rather than relying exclusively on static single-turn tests.

A mature research and engineering discipline for LLM lifecycle management therefore requires a shift from **leaderboard thinking to trajectory thinking**. Instead of asking which model is best at a single point in time, researchers and practitioners should ask how each system evolves across a sequence of releases, which dimensions move together, which regressions persist, which benchmarks remain trustworthy, and how those changes affect real-world reliability.

The most important research opportunity is to establish standardized, reproducible, statistically rigorous **LLM drift observatories**: systems that maintain immutable baselines, generate temporally fresh probes, archive outputs, monitor evaluator reliability, detect changepoints, and connect observed drift to application-level risk. Such infrastructure would turn model updating from a largely opaque lifecycle event into a measurable scientific process. As LLMs increasingly function as infrastructure rather than one-off research artifacts, longitudinal drift tracking should become a core requirement of trustworthy evaluation.

References

Berg-Kirkpatrick, T., Burkett, D., & Klein, D. (2012). An empirical investigation of statistical significance in NLP. *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and*

Computational Natural Language Learning (EMNLP-CoNLL), 995–1005. Association for Computational Linguistics. ACL Anthology: D12-1091. <https://aclanthology.org/D12-1091/>

Belz, A. (2021). Quantifying reproducibility in NLP and ML. *arXiv preprint arXiv:2109.01211*. <https://doi.org/10.48550/arXiv.2109.01211>

Bommasani, R., Liang, P., & Lee, T. (2023). Holistic evaluation of language models. *Annals of the New York Academy of Sciences*, 1525(1), 140–146. <https://doi.org/10.1111/nyas.15007>. PMID: 37230490.

Chen, C., Zhao, Y., Tang, X., Gerstein, M., & Cohan, A. (2024). Investigating data contamination in modern benchmarks for large language models. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics. <https://aclanthology.org/2024.naacl-long.482/>

Chen, J., Dong, Y., Li, H., Su, W., Zhou, Y., Zhang, M., Liu, Y., & Ai, Q. (2026). Benchmarking LLM-as-a-judge for long-form output evaluation. *arXiv preprint arXiv:2606.01629*. <https://doi.org/10.48550/arXiv.2606.01629>.

Chen, L., Zaharia, M., & Zou, J. (2024). How is ChatGPT’s behavior changing over time? *Harvard Data Science Review*, 6(2). <https://doi.org/10.1162/99608f92.5317da47>. arXiv:2307.09009.

Chen, S., Chen, Y., Li, Z., Jiang, Y., Wan, Z., He, Y., Ran, D., Gu, T., Li, H., Xie, T., & Ray, B. (2025). Benchmarking large language models under data contamination: A survey from static to dynamic evaluation. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 10080–10098. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-main.511>.

Choi, H. K., Khanov, M., Wei, H., & Li, Y. (2025). How contaminated is your benchmark? Measuring dataset leakage in large language models with Kernel Divergence. *Proceedings of the 42nd International Conference on Machine Learning, Proceedings of Machine Learning Research*, 267, 10666–10682.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*. <https://doi.org/10.48550/arXiv.2110.14168>.

Deng, C., Zhao, Y., Tang, X., Gerstein, M., & Cohan, A. (2024). Investigating data contamination in modern benchmarks for large language models. *Proceedings of NAACL-HLT 2024*. Association for Computational Linguistics. <https://aclanthology.org/2024.naacl-long.482/>

Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker’s guide to testing statistical significance in natural language processing. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). Measuring massive multitask language understanding. *International Conference on Learning Representations (ICLR 2021)*. arXiv: 2009.03300. <https://doi.org/10.48550/arXiv.2009.03300>.

- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. L., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., & Sifre, L. (2022). An empirical analysis of compute-optimal large language model training. *Advances in Neural Information Processing Systems*, 35. <https://doi.org/10.52202/068431-2176>. arXiv:2203.15556.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., & Stoica, I. (2025). LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. *International Conference on Learning Representations (ICLR 2025)*. arXiv:2403.07974. <https://doi.org/10.48550/arXiv.2403.07974>.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.06770. <https://doi.org/10.48550/arXiv.2310.06770>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*. <https://doi.org/10.48550/arXiv.2001.08361>.
- Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D., Hudson, D. A., Zelikman, E., et al. (2022). Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*. <https://doi.org/10.48550/arXiv.2211.09110>.
- Lin, B. Y., Deng, Y., Chandu, K., Brahman, F., Ravichander, A., Pyatkin, V., Dziri, N., Le Bras, R., & Choi, Y. (2024). WildBench: Benchmarking LLMs with challenging tasks from real users in the wild. *arXiv preprint arXiv:2406.04770*. <https://doi.org/10.48550/arXiv.2406.04770>.
- Qiao, D., Ding, C., & Fan, J. (2026). Mapping and measuring the behavioral evolution of large language models. *arXiv preprint arXiv:2608.11027*. <https://doi.org/10.48550/arXiv.2608.11027>.
- Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are emergent abilities of large language models a mirage? *arXiv preprint arXiv:2304.15004*. <https://doi.org/10.48550/arXiv.2304.15004>.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., & Chen, W. (2024). MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*. <https://doi.org/10.48550/arXiv.2406.01574>.
- Zheng, J., Ritter, A., & Xu, W. (2024). NEO-BENCH: Evaluating robustness of large language models with neologisms. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*, 13885–13906. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.749>.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in*

Neural Information Processing Systems, 36, 46595–46623. <https://doi.org/10.52202/075280-2020>. arXiv: 2306.05685.